

Seguridad y privacidad

Censo Municipal de Animales · Navalcarnero

Seguridad y privacidad

Modelo de roles

Rol	Cómo se identifica	Capacidades clave
Anónimo	sin token	sólo login
Empleado	token de la tabla EMPLEADOS	CRUD propietarios y animales, seguros, bajas
Policía	token de la tabla POLICIA_USUARIOS	búsqueda por chip, registrar incidencias
Administrador	token de empleado + cabecera X-Admin-Access	todo lo anterior + auditoría, backups, gestión usuarios

La cabecera X-Admin-Access la inyecta nginx (o `run_local.py`) en función del puerto: el vhost de admin (8181) la pone a `true`, los demás a vacío. **Nunca confíes en el cliente** para esta cabecera; el cliente no puede sobreescribirla porque nginx la fuerza desde el server-block.

Autenticación

- Contraseñas almacenadas con `bcrypt` (factor por defecto de la librería).
- Tokens de sesión generados con `secrets.token_urlsafe(32)` y persistidos

en `auth.json` (configurable con `AUTH_FILE`). Son revocables eliminando la entrada.

- Endpoints de login devuelven el token al cliente, que lo reenvía en cada

petición.

- Tokens caducan y se renuevan al verificar.

Validación de entrada

- DNI/NIE: helper `_validar_nif` (formato + dígito de control).
- Chip: helper `_validar_chip` (longitud y caracteres).
- Fechas: ISO 8601 obligatorio; rechazo con 400 si no parsea.
- Sexo, gravedad, tipo de incidencia: contra catálogos cerrados.

SQL injection

- Todas las consultas usan parámetros (`%s`) con `mysql.connector`.
- Los nombres de columna que se insertan en SQL son contruidos

dinámicamente sólo a partir de un set conocido (`mapa`), nunca de input de usuario.

Subida de archivos

- Lista blanca de extensiones (.jpg .jpeg .png .gif .webp).
- El nombre del archivo se sanea con regex `[^A-Za-z0-9_-]` antes de

escribir.

- Se almacenan bajo `web/fotos_animales/` y `web/fotos_incidentes/`,

servidas como estáticas (no se ejecutan).

- No se procesa el contenido (no hay redimensionado server-side ni EXIF

rewriting). Si esto cambia, considera librerías como Pillow para evitar parser bugs.

XSS

- El frontend renderiza con plantillas string interpoladas. ****Datos de**

usuario no escapados **llegan a `innerHTML` en varios sitios — eso es aceptable porque sólo usuarios autenticados (empleados/admin/policía) pueden alimentar esos campos, pero conviene migrar a `textContent`**** o a un escapador centralizado en el medio plazo.

- Las cabeceras CSP no están definidas; añadirlas si se expone a internet.

CORS

- `flask_cors.CORS(app)` permite cualquier origen. Apropiado mientras la

API se expone sólo a través del mismo nginx; **restringir** si se separa el front del back.

Auditoría

- Cada operación escribe una línea JSON en `logs/auditoria.jsonl`:
- timestamp UTC
- acción (`crear_animal`, `crear_incidente`, `login`, ...)
- usuario y rol
- identificadores de la entidad afectada
- éxito / error
- El JSONL no contiene datos sensibles (contraseñas, tokens en claro).
- Endpoints `/api/auditoria` y `/api/auditoria/descargar` requieren admin.

Backups

- `mysqldump` programado (default: diario).
- Compresión gzip y rotación con retención configurable.
- Endpoint `restaurar` requiere admin.
- Considera mover backups fuera del host con cron + rsync/SFTP.

RGPD

- Datos personales: DNI/NIE, nombres, teléfonos, email, direcciones.
- Migración 09 introduce campos para marcar registros como purgados.
- El sistema permite ejercer el derecho de supresión: localiza al

propietario y bórralo desde el panel admin (cascade hacia animales y domicilios según FK).

- Conserva el log de auditoría como prueba de cumplimiento, pero

pseudonimiza si se exporta a terceros.

Hardening recomendado en producción

- Servir tras HTTPS (Caddy/Traefik).
- Cambiar `DB_PASSWORD` y la contraseña inicial del admin.
- Restringir el panel :8181 por VPN / firewall.
- Añadir CSP estricta y `X-Frame-Options: DENY`.
- Rate-limit en `/api/auth/login` (fail2ban o nginx `limit_req`).
- Auditar `auth.json` regularmente: revocar tokens viejos.
- Backups verificados (restaurar a entorno separado periódicamente).